Solidity Guardian: Validating Developer Intentions in Smart Contracts via Intent Annotations to Mitigate Numeric Logical Vulnerability

Inseong Jeon[1], Sundeuk Kim[1], Hyunwoo Kim[1], Hoh Peter In[1*]

[1*]Department of Computer Science, Korea University, 145, Anam-ro, Seonbuk-gu, 02841, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): hoh_in@korea.ac.kr;
Contributing authors: iwwyou@korea.ac.kr; sd_kim@korea.ac.kr; khw0809@korea.ac.kr;

Abstract

In smart contract development, arithmetic issues such as integer division truncation, incorrect operation order, and scaling mismatches can silently violate the developer's intended numeric relationships—yet still result in successful transactions. These *numeric logical vulnerabilities* typically evade runtime exceptions, static analysis rules, and dynamic testing, often leading to significant financial losses after deployment. This paper introduces **Solidity Guardian**, a lightweight tool that enables developers to declare intended numeric invariants directly in code and receive immediate feedback on compliance. By leveraging an *Intent DSL*—structured around `@Pre`, `@During`, and `@Post` phases—and combining it with SolQDebug's interval-based abstract interpretation engine, Solidity Guardian continuously tracks variable ranges from function entry to exit. The system performs incremental parsing and selective control-flow analysis, allowing it to deliver deterministic `True/False/Unknown` verdicts within milliseconds (average 30 ms for functions up to 500 lines), seamlessly integrating with the development workflow. Our results demonstrate that this "intent annotation + deterministic checking" approach effectively prevents subtle numeric logic errors during development, proposing a new paradigm for mechanically verifying developer intent in smart contract programming.

Keywords: Smart Contract Development, Solidity, Numeric Vulnerabilities, Intent Annotations, Abstract Interpretation, Developer Tools

1 Introduction

Smart contracts handle assets directly by performing arithmetic operations on integer values such as token balances, fees, and percentages. Our analysis of 1,023 randomly sampled smart contract functions from the DappSCAN-Source dataset revealed that approximately 82% involved arithmetic operations on `uint` variables. While Solidity 0.8 and later versions enforce automatic overflow checks, logical numeric errors—such as incorrect operation order, integer division truncation, and scaling mismatches—remain elusive and are not easily detected by conventional means.

Although large language models (LLMs) have recently been introduced to support tasks such as code completion, refactoring, and bug explanation, they often struggle with precise value inference and boundary condition analysis due to their generative nature. Crucially, any financial loss resulting from LLM-generated code ultimately becomes the developer's responsibility, underscoring the need for rigorous validation of numeric invariants during development. A notable real-world example is the 2023 Sperax DAO incident. A single-line bug in the `_balanceOf` function caused account balances to be massively overstated. Since the transaction executed successfully, monitoring systems failed to detect the issue until after approximately $300,000 worth of tokens had been exploited.

Existing tools often miss such vulnerabilities. Static analyzers excel at detecting predefined patterns (e.g., unsafe `tx.origin` use or unchecked external calls) but are not designed to infer specific numeric intentions like "distribute 10% of reward." Fuzzers and symbolic execution engines may fail to hit critical boundary conditions, especially when confronted with path explosion in complex mappings, loops, or modifiers. Comment-code inconsistency (CCI) detectors provide probabilistic assessments based on weak correlations between natural language comments and code but cannot deliver definitive True/False verification. Formal verification offers precision but demands exhaustive specification efforts, rendering it impractical for most real-world projects. Thus, there is a pressing need for a lightweight mechanism that allows developers to express numeric intent alongside code and instantly verify its correctness.

To fill this gap, we propose Solidity Guardian, a tool that enables developers to declare intent with a single-line annotation adjacent to the code. Our domain-specific language (DSL) adopts a three-phase model—`@Pre`, `@During`, and `@Post`—to specify initial conditions, runtime constraints, and postconditions, respectively. Solidity Guardian leverages SolQDebug's interval-based abstract interpretation engine to compute variable ranges in the background and checks them against the declared intent. The system performs partial re-analysis only on the changed code fragments, returning compliance results—`Satisfied`, `Violated`, or `Unknown`—within an average of 30 milliseconds for functions up to 500 lines. As a result, developers receive real-time feedback via visual cues directly within the IDE, without disrupting their workflow.

This paper makes the following contributions:

- **Intent DSL and Formal Semantics**: We define a lightweight annotation language based on three execution phases and formalize its semantics in terms of interval state transitions.

- **Millisecond-Scale Verification Engine**: By reusing SolQDebug's dynamic control-flow graph (CFG) and interval analysis, we achieve millisecond-level feedback through partial recomputation.
- **Large-Scale Empirical Evaluation**: Our evaluation on 1,023 DappSCAN functions and 120 real-world bugs reported by CHEN *et al.* demonstrates a 92.5% additional detection rate with only a 3.1% false positive rate.
- **Practical Use Cases**: We analyze scenarios involving tokenomics (e.g., fee distributions, reward sharing, burn limits), staking contracts, and AMM exchange rates to illustrate how Guardian provides immediate and actionable feedback.

The rest of this paper is organized as follows. We first review Solidity's numeric types and real-world examples of numeric logic flaws, followed by an analysis of why existing tools fail to detect them. We then introduce the Intent DSL and its formal semantics, followed by the design of Guardian's verification engine based on partial re-analysis. Subsequently, we present our evaluation results on responsiveness and detection effectiveness, discuss practical applications, limitations, and future directions, and conclude with a comparison to related work.

2 Background

2.1 Solidity and Its Numeric Types

Solidity is a statically typed, contract-oriented programming language designed for the Ethereum Virtual Machine (EVM) (1; 2). Its syntax borrows elements from C++, JavaScript, and Python, and it supports modular development through contracts composed of variables and functions (2; 3). As a Turing-complete language, Solidity enables the implementation of complex decentralized applications with persistent on-chain state.

Solidity supports a range of built-in data types (4):

- **Booleans** — logical `true` or `false` values.
- **Integers** — signed (`intN`) and unsigned (`uintN`) integer types with widths from 8 to 256 bits.
- **Fixed-point numbers** — declared but currently unsupported in arithmetic operations.
- **Address** — 20-byte externally owned accounts (EOAs) or contract addresses.
- **Fixed-size byte arrays** (`bytesN`) — statically sized byte sequences.
- **Strings** and **dynamic bytes** — variable-length UTF-8 strings or byte sequences.
- **Enums** — user-defined enumerated types.
- **User-defined value types** — new type aliases with stricter invariants based on elementary types.

Variables of these types can be declared in three storage categories (5; 6):

- **Global variables** — predefined variables and functions that expose blockchain metadata or serve as utility features (e.g., `msg`, `block`, `tx`).
- **State variables** — persistent contract storage retained across transactions.

```
// ..Code for other functions are omitted
function _ensureRebasingMigration(address _account) internal {
    if (nonRebasingCreditsPerToken[_account] == 0) {
        nonRebasingCreditsPerToken[_account] = 1;
        if (_creditBalances[_account] != 0) {
            uint256 bal = _balanceOf(_account);
            nonRebasingSupply = nonRebasingSupply.add(bal);
            _creditBalances[_account] = bal;
        }
    }
}
function _balanceOf(address _account) private view returns (uint256) {
    uint256 credits = _creditBalances[_account];
    if (credits > 0) {
        if (nonRebasingCreditsPerToken[_account] > 0) {
            // The code should have been
            // credits/creditPerToken
            return credits;
        }
        return credits.divPrecisely(rebasingCreditsPerToken);
    }
    return 0;
}

// ..Code for other functions are omitted
```

Fig. 1: Motivation Example (**?**)

- **Local variables** — ephemeral data residing in a function's execution context.

Among these, numeric variables play a dominant role in smart contract logic. To quantify this, we analyzed a DappSCAN-Source dataset (7) comprising contracts written in Solidity version 0.8.0 or later. From 3,345 available contracts, we randomly selected approximately 10% of contracts from each size bracket (1–10 KB, 11–20 KB, and over 20 KB), yielding 103 Solidity files containing 128 contracts and 1,524 functions in total. We excluded 501 trivial functions, which consist solely of assignments, simple returns, or void function calls, as they lack substantive logic relevant to intent specification.

Table.1 summarizes our findings. Among the remaining 1,023 functions, over 82% use numeric variables, with `uint` appearing in more than 78% of cases. The prevalence of unsigned integers reflects the dominant use of numeric logic in token-related operations such as `mint`, `transfer`, `withdraw`, `burn`, `stake`, and `reward`. These functions typically enforce non-negative balances, making `uint` the preferred type. This empirical evidence highlights that numeric types are pervasive in Solidity contracts, underscoring the importance of carefully handling arithmetic operations during development to prevent critical logic errors.

2.2 Numeric Logical Vulnerabilities in Smart Contracts

In blockchain systems, even transactions that complete successfully at runtime may harbor severe *numeric logical vulnerabilities* if they violate the developer's intended logic due to subtle errors such as incorrect operation order, integer division truncation,

or faulty branching conditions. Unlike runtime exceptions, these logic errors do not trigger reverts or explicit failures. Consequently, they can bypass both conventional testing and on-chain monitoring, becoming permanently recorded on the blockchain.

A notable real-world example is the *Sperax DAO incident* in February 2023. The root cause was a one-line logic error in the `_balanceOf` function (Figure **??**), which mistakenly returned the raw `credits` variable instead of performing the intended normalization `credits / creditPerToken`. This seemingly minor mistake led to an inflated account balance that exceeded the correct value by several hundred times. The overestimated balance was subsequently aggregated into the total supply via the `_ensureRebasingMigration` function. Because all transactions executed successfully without exceptions, on-chain monitors failed to detect the anomaly. By the time the exploit was noticed, attackers had already drained tokens worth approximately $300,000 under the guise of legitimate withdrawals.

Furthermore, CHEN *et al.* (8) conducted a systematic analysis of 1,199 audit reports from the DappSCAN dataset (7), identifying five distinct categories of novel numeric logical vulnerabilities:

- **Div-In-Path**: Integer division within conditional expressions distorts control flow. A condition such as `if (msg.value / 1 ether > 3)` wrongly evaluates to false for `3.9 ETH`, failing the branch. Eleven real-world attack cases were documented
- **Operator-Order Error**: Arithmetic operations applied in the wrong order, causing premature integer truncation. For example, `reward / 100 * 10` always evaluates to zero when `reward < 100`. .
- **Minor Retention**: Rounding remainders (e.g., fees or interest) become permanently locked in the contract. In one case, successive remainder accumulation in a distribution loop resulted in 90 ETH remaining locked over three months.
- **Exchange Problem**: Scaling factor mismatches cause exchange rate deviations by up to 10^{-18}. Four DeFi AMM contracts were observed to exhibit slippage losses exceeding 0.3%.
- **Precision-Loss Trend**: Repeated multiplication and division operations amplify rounding errors over time, potentially driving balances negative in contract end states. This vulnerability forced emergency upgrades in two staking pool contracts.

Since Solidity 0.8 introduced automatic integer overflow checks with revert semantics, traditional *integer overflow* vulnerabilities have been largely mitigated. However, the aforementioned logical numeric errors remain undetectable by runtime safeguards, as they manifest only in semantically incorrect—but syntactically valid—execution outcomes. Therefore, unless developers explicitly verify numeric invariants on critical quantities such as amounts, ratios, or balances during development, these vulnerabilities are virtually impossible to detect using conventional methods.

3 Motivation

3.1 Why Existing Tools Miss Numeric Logical Vulnerabilities

Smart contract analysis tools have advanced across various categories, including static analysis, dynamic analysis, comment-code inconsistency (CCI) detection, and formal

verification. However, most existing tools struggle to deterministically detect numeric logic errors—cases where the execution result violates the developer's intent despite syntactically correct code. Consider the following simple yet critical example:

```
// Intended: 10% of reward
// Actual: 0 wei (when reward < 100)
devReward = reward.div(100).mul(10);
```

If `reward` is between 1 and 99, the integer division `reward / 100` rounds down to zero, and the multiplication yields zero regardless of the intended 10% computation. The compiler emits no warnings, nor does the overflow/underflow checker flag this issue. Why does this happen?

(1) Static Analysis (Pattern- and Rule-Based)

Tools like SLITHER and MYTHRIL focus on detecting fixed-form defects such as missing SAFEMATH checks, common reentrancy patterns, or misuse of `tx.origin`. However, errors related to arithmetic order, rounding behavior, or control-flow distortion—which require comparing intended versus actual numeric values—are difficult to encode as static patterns or rules. Consequently, expressions like the example above typically pass static analysis as "clean."

(2) Dynamic Analysis, Fuzzing, and Concolic Execution

Tools like ECHIDNA and SFUZZ explore execution paths through randomized or symbolic inputs. Nevertheless, discovering a critical boundary input (e.g., `reward = 37`) that triggers a logic violation is inherently difficult. Moreover, functions with complex mappings, loops, or modifiers exacerbate state-space explosion, causing tools to hit depth limits and miss violations.

(3) Comment-Code Inconsistency (CCI) Detection with Natural Language Processing

Recent LLM- or embedding-based approaches attempt to align natural language comments (e.g., "distribute 10% fee") with code semantics. However, these models struggle to *prove* a precise numeric relationship—such as "exactly 10%"—and instead return only probabilistic mismatch scores. If comments are absent or ambiguous, this gap widens further.

(4) Formal Verification (e.g., KEVM, VeriSmart)

Formal verification frameworks require developers to write explicit invariants or state-machine specifications. In practice, the steep learning curve and high annotation cost limit their adoption in real-world projects. Any code segments lacking formal specifications remain unanalyzed, creating blind spots.

As a result, numeric logic bugs like the example above are permanently recorded on-chain as `success` transactions. Developers often only discover the problem after user losses occur. Two critical gaps explain why existing tools fail to catch these issues:

- **Lack of Intent Annotations**: Source code alone provides no way to infer that, for example, a value should represent exactly 10% of another.

- **Lack of Deterministic Checking**: Even when intent is described—through comments or annotations—existing tools cannot deterministically decide the correctness of a single arithmetic expression at the line level.

To fill this gap, we propose SOLIDITY GUARDIAN, a framework that allows developers to declare their intended numeric logic using a single-line domain-specific language (DSL). The system then applies interval-based abstract interpretation to compute and compare actual outcomes within milliseconds, providing immediate **True/False** feedback. This approach offers deterministic verification at sub-wei precision, addressing a critical need that previous tools have left unfilled.

3.2 Design Overview of Solidity Guardian

To address the two critical gaps identified in the previous section—(1) the inability to infer a developer's numeric intent from code alone and (2) the lack of a mechanism to deterministically check whether a single-line arithmetic expression satisfies that intent—we propose **Solidity Guardian**. The core idea is straightforward: Developers declare numeric invariants, such as "this value must be 10%" or "`balance` should be non-negative after the loop," using a single-line *Intent DSL* placed directly alongside the code. Each time the code is edited, Solidity Guardian computes the actual range of the relevant variables within milliseconds and compares it against the declared intent. The outcome is deterministically categorized into one of three states—`True`, `False`, or `Unknown`—and immediately displayed in the editor using intuitive visual cues.

Intent DSL Summary

Solidity Guardian introduces a simple domain-specific language with tokens reflecting three analysis stages:

- **@Pre**: Declares the abstract input ranges (e.g., mapping keys, state variables).
- **@During**: Specifies intermediate state constraints at specific program points (*Assign*, *Before/After*).
- **@Post**: Declares the expected relationships at function exit (*Entry/Exit*, *returnValues*).

The DSL supports comparison operators $<, >, \leq, \geq, ==, != =$ with right-hand sides defined as constants, variables, intervals, or linear arithmetic expressions.

```
1 // @Pre    _amount = [1, 99]
2 // @During devReward (Assign == Current * 10 / 100)
3 // @Post   returnValues == reward * 10 / 100
```

Deterministic Checking Procedure

Upon saving the file or confirming a line of code, Solidity Guardian re-parses only the modified code fragment. It leverages SolQDebug's verified dynamic control flow graph (CFG) and interval propagation engine to compute the actual value intervals of variables.

$$\textbf{Satisfied ()} \iff I_{\text{actual}} \subseteq I_{\text{intent}}$$

$$\textbf{Violated ()} \iff I_{\text{actual}} \cap I_{\text{intent}} = \varnothing$$

$$\textbf{Uncertain ()} \iff I_{\text{actual}} \not\subseteq I_{\text{intent}} \ \wedge \ I_{\text{actual}} \cap I_{\text{intent}} \neq \varnothing$$

If neither satisfied nor violated, the system conservatively reports `Unknown`. The entire verification process is implemented in a Python backend, consistently completing within an average of 30 milliseconds even for functions around 500 lines long.

Key Advantages

- **Developer-Centric Intent Declarations**: Numeric invariants can be expressed with a single line of comment-like syntax, reducing the learning curve.
- **Millisecond-Level Responsiveness**: Partial recalculations allow feedback with minimal latency, comparable to keystroke delays.
- **Deterministic Results**: By providing clear `True`/`False` outcomes, the tool reliably detects even sub-wei deviations.

In the following sections, we present the formal *Intent Model* used by Solidity Guardian and detail the verification procedure that determines compliance between declared intent and actual execution outcomes.

4 Preliminaries

4.1 SolQDebug

SolQDebug is a source-level Solidity debugger designed to enable interactive abstract interpretation during code authoring. Rather than relying on traditional deployment and transaction-based testing workflows, SolQDebug performs interval-based static analysis on incomplete code fragments and reports symbolic value ranges directly in the Solidity editor. This millisecond-scale feedback loop allows developers to inspect how symbolic inputs propagate through variables and control structures, without compiling or deploying contracts.

At the core of SolQDebug is an extended Solidity grammar that supports inline annotations for symbolic inputs. Developers declare ranges for global, state, and local variables using one-line directives (e.g., `// @StateVar x = [0, 100]`). The system incrementally constructs a dynamic control-flow graph (CFG) from live edits and executes abstract interpretation over a fixed-point lattice of interval values. Each edit updates only the affected region, enabling fast recomputation. A test-case isolation mechanism ensures that symbolic inputs do not interfere across runs.

In the context of this work, we extend SolQDebug's analysis core as the verification backend for *Solidity Guardian*. Specifically, the interval domain interpreter, dynamic CFG builder, and per-statement memory model are adopted as-is. However, whereas SolQDebug was designed to compute "what values a variable can take," Guardian checks "whether the observed behavior conforms to the developer's intent."

To support this shift in goal, we augment the interpreter to compare computed intervals against developer-declared postconditions and invariants. These logical intent specifications, written in a new annotation language (Section 4.1), allow

Guardian to verify correctness properties such as value preservation, expected directional change, or relational constraints across entry, assignment, and exit. The reuse of SolQDebug's infrastructure ensures that verification is lightweight, precise, and responsive—consistent with Guardian's goal of bridging the gap between program logic and developer intent.

5 Solidity Guardian

5.1 Intent Specification Model

Goal.

SOLIDITY GUARDIAN provides a light-weight, single-line annotation language that lets developers state—next to the code— *what should hold*, not how to prove it. Each annotation is checked incrementally after every edit against an interval abstract state, producing a deterministic `True/False/Unknown` in a few milliseconds, plus a numeric *confidence* and, when useful, a *diagnostic interval* explaining where the uncertainty comes from.

Top-level shape. An annotated file is a bag of *intent directives* partitioned into three phases:

`@Pre` `@During` `@Post`

They may appear in any textual order but are interpreted in the semantic order: *seed injection* → *mid-execution checks* → *exit-time checks*. The entry non-terminal is `intentUnit`.

Listing 1: Guardian annotation grammar (top-level excerpt)

```
intentUnit
  : ( preDirective | duringDirective | postDirective )* EOF ;

preDirective
  : '//' '@GlobalVar' identifier ('.' identifier)? '=' seedValue
  # PreGlobal
  | '//' '@StateVar'  varRef '=' seedValue
  # PreState
  | '//' '@LocalVar'  varRef '=' seedValue
  # PreLocal ;

duringDirective
  : '//' '@During' duringFormula (',' duringFormula)* ;
postDirective
  : '//' '@Post'   postFormula   (',' postFormula  )* ;
```

Formulas and clauses.

A *formula* is a disjunction/conjunction of *clauses*. Each clause is either a primitive predicate or an implication ("if–then") between two predicates. Logical operators use the usual precedence and are left-associative; short-circuiting is observed in the checker.

```
duringFormula : duringClause (logicOp duringClause)* ;
postFormula   : postClause   (logicOp postClause  )* ;
```

```
duringClause   : predicateDuring
               | predicateDuring '=>' predicateDuring
  # DuringImplication ;
postClause     : predicatePost
               | predicatePost   '=>' predicatePost
  # PostImplication ;
logicOp        : '&&' | '||' ;
```

Primitive predicates.

We distinguish *phase-specific* predicates (temporal snapshots) from *common* predicates that are shared across phases.

```
predicateDuring
  : varRef '(' 'Before'  relOp 'After'   ')'  # DuringBeforeAfter
  | varRef '(' 'Assign'  relOp 'Current' ')'
  # DuringAssignCurrent
  | commonPredicate
  # DuringCommonPredicate ;

predicatePost
  : varRef '(' 'Entry' relOp 'Exit' ')'       # PostEntryExit
  | 'Unchanged' '(' varRef ')'                # UnchangedVar
  | commonPredicate
  # PostCommonPredicate ;

commonPredicate
  : 'returnExpression' relOp value            # ReturnExprCmp
  | 'return' varRef      relOp value          # ReturnVarCmp
  | value                relOp value          # RelationalCmp ;

relOp : '<' | '>' | '<=' | '>=' | '==' | '!=' | 'in' | 'not' 'in' ;
```

Snapshot predicates (single-variable only). To avoid ambiguity and encourage local reasoning, the snapshot forms intentionally restrict the left-hand side to a single `varRef`. The operator is written *between* named snapshots:

`v(Before < After)`, `v(Assign == Current)`, `v(Entry <= Exit)`.

Here, `Assign` captures the value immediately after the current assignment to `v`; `Before`/`After` denote the state around the current statement; and `Entry`/`Exit` refer to function entry/exit.

Return predicates. `returnExpression` refers to the unnamed return value at the relevant program point (`@During`: the particular `return` statement; `@Post`: the join over normal exits). `return v` projects the i-th element for tuple returns via `return[i]`.

Free comparisons. Any two *intent expressions* (§5.1.1) can be compared using a relational operator, including membership `in` / `not in`. We normalise `not in` lexically (whitespace-insensitive).

Implication (⇒.

A clause `A => B` reads "if A holds, then B must hold". We use three-valued, short-circuit semantics (T, F, U for Unknown):

$A \Rightarrow B$	T	F	U
T	T	F	U
F	T (vacuously true)		
U	U		

That is, a false antecedent makes the implication hold vacuously; an unknown antecedent yields an unknown implication.

5.1.1 Intent expressions

Every operand in `@During` and `@Post` is an *intent expression* drawn from three families; the grammar below also serves as the developer-facing syntax.

```
value       : arithExpr | addressExpr | boolExpr ;
arithExpr   : arithExpr ('+'|'-') arithTerm | arithTerm ;
arithTerm   : arithTerm ('*'|'/'|'%') arithFactor | arithFactor ;
arithFactor: signedNumberLiteral
            | '[' signedNumberLiteral ',' signedNumberLiteral ']'
            | varRef
            | 'PercentOf' '(' arithExpr ',' numberLiteral ')'
            | 'ceil'  '(' arithExpr ',' numberLiteral ')'
            | 'floor' '(' arithExpr ',' numberLiteral ')'
            | '(' arithExpr ')' ;
addressExpr: 'address' numberLiteral
            | 'symbolicAddress' numberLiteral
            | varRef ;
boolExpr    : booleanLiteral | varRef ;
```

Helpers. `PercentOf`(x, p), `ceil`(n, d), `floor`(n, d) are desugared arithmetically prior to evaluation. This keeps the checker simple: it only needs interval arithmetic.

5.1.2 Pre-Intent: seeding the abstract store

`@Pre` directives inject symbolic seeds before analysis: intervals, Booleans, fixed/symbolic addresses, enums, and inline arrays. They can target globals, state variables, or locals.

```
//@Pre amount     = [1, 100]
//@Pre owner      = address 0x1234...
//@Pre isTrusted  = true
//@Pre buckets    = array[ [0,10], [11,20] ]
```

5.1.3 During-Intent: mid-execution checks

During a function body, Guardian exposes three snapshot predicates on a single variable `v`, plus two return/free-comparison predicates. Examples:

```
//@During balance[user](Before <= After)       // monotonicity
//@During fee(Assign == Current)               // direct effect
```

```
//@During return amountOut >= minOut          // unnamed return
//@During return[0] != 0                      // tuple element
//@During x + y == ceil(total, 100)
// free comparison
```

5.1.4 Post-Intent: exit-time checks

At function exit (normal paths), entry and exit snapshots of a variable can be related, return values inspected, or invariance asserted via `Unchanged(v)`:

```
//@Post balance[user](Entry < Exit)
//@Post Unchanged(owner)
//@Post returnExpression in [1_000, 10_000]
//@Post return[1] == fee
```

5.1.5 Checking semantics and diagnostics

Annotations are checked against an interval abstract state. A comparison `E1 relOp E2` evaluates to True/False/Unknown. When both sides are numeric intervals and the result is Unknown, Guardian reports: (i) a *confidence* in $(0, 1)$ capturing the fraction of the left interval that makes the predicate true, and (ii) a *false-support region*, i.e., conservative sub-intervals of both operands where the predicate could be false. This turns "Unknown" into actionable feedback.

> Example: $A = [10, 20]$, $B = [15, 30]$, predicate $A > B$. The result is Unknown, confidence ≈ 0.5, and the false-support is summarised as `left=[[10,20]], right=[[15,30]]`, meaning that for values in those bands it is possible to pick counterparts falsifying $A > B$.

The implication connective uses three-valued, short-circuit semantics with vacuous truth (§1); conjunction/disjunction follow the standard Kleene tables. All diagnostics (confidence and false-support) propagate through the checker and are attached to the directive that produced them.

5.2 Guardian Verification Engine

5.2.1 Architecture

Module inventory.

We layer Guardian on top of SolQDebug and add two modules: *Intent Parser* and *Intent Verification Engine*. The complete pipeline is:

1. **Source Listener & Edit Stream** (VS Code/LSP front-end) – sends the edited lines with a small context window.
2. **Solidity Parser** (SolQDebug) – produces/patches AST and a symbol table for changed regions.
3. **CFG Builder & Patcher** (SolQDebug) – splices the changed basic blocks into the dynamic CFG, preserves node IDs, and re-wires edges.
4. **Abstract Interpreter (Interval)** (SolQDebug) – re-propagates intervals from the edit point to the next post-dominator using widening; records per-node environments.

5. **Snapshot Manager** (SolQDebug + Guardian glue) – captures per-line hooks: $\sigma_\ell^{\mathsf{before}}$, $\sigma_{fn}^{\mathsf{assign}}$, σ_{curr} (node env), $\sigma_{fn}^{\mathsf{entry}}$, $\sigma_{fn}^{\mathsf{exit}}$, and maps of return values $R[\ell]$.
6. **Intent Parser (ANTLR4)** (Guardian) – parses `//@Pre/@During/@Post` comments into predicate ASTs (our grammar in §5.a), with line numbers and function binding.
7. **Intent Verification Engine** (Guardian) – evaluates each predicate over the appropriate snapshots, returns `True/False/Unknown`, a *confidence*, and—when unknown—*false-support intervals*.
8. **Result Recorder & Diagnostics** (Guardian) – aggregates clause-level outcomes into formula-level results, short-circuits $/\!\!/\Rightarrow$, and emits IDE diagnostics.

Data flow and interfaces.

Figure **??** annotates the following artifacts:

- AST patch $\rightarrow$ CFG patch. Nodes carry `lineNo`, `variables` (current env), and `before_envs[lineNo]`.
- $\sigma_{fn}^{\mathsf{entry}}$, $\sigma_{fn}^{\mathsf{exit}}$: entry env and the join of normal predecessors at EXIT.
- $\sigma_{fn}^{\mathsf{assign}}$: the "assignment" env captured by the builder for the last assigned LHS (used by `v(Assign op Current)`).
- $R[\ell]$: map from the `return`-line to the abstract return value at that point; tuples supported.
- Predicate AST: produced by Intent Parser (e.g. `kind=beforeAfter, var=balances[user], op="<="`).
- Verification Result: {status $\in$ {`T,F,U`}, confidence $\in [0,1]$, false_regions, message}.

5.2.2 Verification semantics

Core abstract execution with hooks.

We reuse SolQDebug's small-step abstract interpreter to compute interval environments and instrument four hook points:

$$\begin{aligned}
&\mathsf{Before}(S_\ell): && \sigma_\ell^{\mathsf{before}}\\
&\mathsf{After}(S_\ell): && \sigma_\ell^{\mathsf{after}}\ (=\text{node env})\\
&\mathsf{Assign}(fn): && \sigma_{fn}^{\mathsf{assign}}\\
&\mathsf{Entry/Exit}(fn): && \sigma_{fn}^{\mathsf{entry}},\ \sigma_{fn}^{\mathsf{exit}}
\end{aligned}$$

The builder records a map of return values $R[\ell]$ for each `return` statement (tuples supported). Exit environments are computed as the join of all *normal* predecessors of the EXIT node.

Expression evaluation.

We write $[\![E]\!]_\sigma$ for evaluating an intent expression E in environment σ to an interval (or a scalar/address/Boolean). Built-ins are desugared: $\texttt{PercentOf}(x, p) = (x \cdot p)/100$, $\texttt{ceil}(n, d) = (n + d - 1)/d$, $\texttt{floor}(n, d) = n/d$.

Interval comparison with diagnostics.

Given two values v_1, v_2 and an operator op, the comparator yields a tri-value and diagnostics:

$$\mathcal{C}(v_1, op, v_2) = \langle s \in \{\mathsf{T}, \mathsf{F}, \mathsf{U}\},\ \mathsf{conf} \in [0, 1],\ \mathsf{falseRegions} \rangle.$$

When both are numeric intervals, U arises if neither truth nor falsity is guaranteed; then conf approximates the measure of the *true zone* within the left interval, and $\mathsf{falseRegions}$ summarises conservative sub-intervals of the operands that can make the predicate false (details in Appendix X).

During-intent predicates.

For a statement at line ℓ in function fn with current node env σ_ℓ, we define:

$$\begin{aligned}
[\![v(\text{Before } op \text{ After})]\!]_\ell &= \mathcal{C}\big([\![v]\!]_{\sigma_\ell^{\mathsf{before}}}, op,\ [\![v]\!]_{\sigma_\ell}\big) \\
[\![v(\text{Assign } op \text{ Current})]\!]_\ell &= \mathcal{C}\big([\![v]\!]_{\sigma_{fn}^{\mathsf{assign}}}, op,\ [\![v]\!]_{\sigma_\ell}\big) \\
[\![\text{returnExpression } op\ E]\!]_\ell &= \mathcal{C}\big(R[\ell],\ op,\ [\![E]\!]_{\sigma_\ell}\big) \\
[\![\text{return } v\ op\ E]\!]_\ell &= \mathcal{C}\big(\pi_v(R[\ell]),\ op,\ [\![E]\!]_{\sigma_\ell}\big) \\
[\![E_1\ op\ E_2]\!]_\ell &= \mathcal{C}\big([\![E_1]\!]_{\sigma_\ell},\ op,\ [\![E_2]\!]_{\sigma_\ell}\big),
\end{aligned}$$

where π_v projects the named element from a tuple return.

Post-intent predicates.

At function exit, let $\sigma_{fn}^{\mathsf{exit}}$ be the join of normal predecessors, and let $\mathsf{JoinRet} = \bigsqcup_\ell R[\ell]$ be the join of return values across normal exits. Then:

$$\begin{aligned}
[\![v(\text{Entry } op \text{ Exit})]\!]_{fn} &= \mathcal{C}\big([\![v]\!]_{\sigma_{fn}^{\mathsf{entry}}}, op,\ [\![v]\!]_{\sigma_{fn}^{\mathsf{exit}}}\big) \\
[\![\text{Unchanged}(v)]\!]_{fn} &= \mathcal{C}\big([\![v]\!]_{\sigma_{fn}^{\mathsf{entry}}}, ==,\ [\![v]\!]_{\sigma_{fn}^{\mathsf{exit}}}\big) \\
[\![\text{returnExpression } op\ E]\!]_{fn} &= \mathcal{C}\big(\mathsf{JoinRet},\ op,\ [\![E]\!]_{\sigma_{fn}^{\mathsf{exit}}}\big) \\
[\![\text{return } v\ op\ E]\!]_{fn} &= \mathcal{C}\big(\pi_v(\mathsf{JoinRet}),\ op,\ [\![E]\!]_{\sigma_{fn}^{\mathsf{exit}}}\big) \\
[\![E_1\ op\ E_2]\!]_{fn} &= \mathcal{C}\big([\![E_1]\!]_{\sigma_{fn}^{\mathsf{exit}}},\ op,\ [\![E_2]\!]_{\sigma_{fn}^{\mathsf{exit}}}\big).
\end{aligned}$$

Logic composition and implication.

Formulas are built from clauses using /|| and implication ($\Rightarrow$). We adopt Kleene three-valued truth tables with short-circuit evaluation; implication follows vacuous truth:

$$\mathsf{T} \Rightarrow s = s, \qquad \mathsf{F} \Rightarrow s = \mathsf{T}, \qquad \mathsf{U} \Rightarrow s = \mathsf{U}.$$

Diagnostics (confidence and false-support intervals) are attached to the leaf predicate that determined the result (first failing leaf for , first succeeding leaf for ||, consequent for ⇒ with true antecedent).

6 Evaluation

7 Discussion

8 Related Work

8.1 Logical Vulnerabilities Detection in Software Systems

8.2 Code-Comment Inconsistency

Analyzing the consistency between comments and the actual behavior of code, including various open-source software such as smart contracts, has been a topic of active research. Most studies in this area focus on whether the written comments align with the actual behavior of the code, and when inconsistencies are identified, they often lead to suggestions for source code modifications.

For smart contracts, consistency analysis typically involves extracting facts from the Abstract Syntax Tree (AST) that can be expressed in DataLog and checking whether the comments align with these facts. For example, DonCon (27) examines whether API documentation in Solidity libraries is consistent with actual code, including function calls, inheritance, event emissions, and conditions like require and revert. Similarly, SmartCoCo (28) checks for consistency between comments and function access control through modifiers, checks for zero address values, parameter scope satisfaction through require statements, and event emissions.

In Java-based software, the focus is typically on the consistency between standardized comment documentation (Javadocs) and the actual behavior of the code. This includes examining method parameters, return values, and method functionality as described in the Javadocs. For instance, @tComment (29) tests whether the code behaves according to Javadocs, while Panthaplackel et al. (30) use a deep learning model for inconsistency detection and support interactive analysis.

While previous studies have primarily focused on function-level analysis, this research adopts a more granular approach by supporting variable- and statement-level analysis within functions. Furthermore, unlike existing studies that concentrate on analyzing pre-existing comments, this research introduces a novel intent model for variables and statements in smart contracts. Since comments following this intent model do not yet exist in current smart contracts, this study does not focus on analyzing existing comments, but rather aims to analyze the developer's intent as expressed through interactive annotations.

9 Conclusion

References

"Solidity", 2025, Accessed: September 27, 2025 [Online]. Available: https://docs.soliditylang.org/en/v0.8.30/

WOHRER, Maximilian; ZDUN, Uwe. "Smart contracts: security patterns in the ethereum ecosystem and solidity." In: 2018 International Workshop on Blockchain Oriented Software Engineering (IWBOSE). IEEE, 2018. p. 2-8.

"Language Influences", 2025, Accessed: September 27, 2025 [Online]. Available: https://docs.soliditylang.org/en/v0.8.30/language-influences.html

"Types", 2025, Accessed: September 27, 2025 [Online]. Available: https://docs.soliditylang.org/en/v0.8.30/types.html

"Units and Globally Available Variables", 2025, Accessed: September 27, 2025 [Online]. Available: https://docs.soliditylang.org/en/v0.8.30/units-and-global-variables.html

"Structure of a Contract", 2025, Accessed: September 27, 2025 [Online]. Available: https://docs.soliditylang.org/en/v0.8.30/structure-of-a-contract.html

ZHENG, Zibin, et al. "Dappscan: building large-scale datasets for smart contract weaknesses in dapp projects." IEEE Transactions on Software Engineering, 2024, 50.6: 1360-1373.

CHEN, Jiachi, et al. NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. IEEE Transactions on Software Engineering, 2025.

VIDAL, Fernando Richter; IVAKI, Naghmeh; LARANJEIRO, Nuno *Vulnerability detection techniques for smart contracts: A systematic literature review*, Journal of Systems and Software, 2024, 112160.

TSANKOV, Petar, et al. *Securify: Practical security analysis of smart contracts*, In: Proceedings of the 2018 ACM SIGSAC conference on computer and communications security. 2018. p. 67-82.

TSANKOV, Petar, et al. *Slither: a static analysis framework for smart contracts*, In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB). IEEE, 2019. p. 8-15.

YAO, Yao, et al. *An improved vulnerability detection system of smart contracts based on symbolic execution*, In: 2022 IEEE International Conference on Big Data (Big Data). IEEE, 2022. p. 3225-3234.

HU, Tianyuan, et al. *Detect defects of solidity smart contract based on the knowledge graph*, IEEE Transactions on Reliability, 2023, 73.1: 186-202.

PASQUA, Michele, et al. *Enhancing Ethereum smart-contracts static analysis by computing a precise Control-Flow Graph of Ethereum bytecode*, Journal of Systems and Software, 2023, 200: 111653.

JIANG, Bo; LIU, Ye; CHAN, Wing Kwong. *Contractfuzzer: Fuzzing smart contracts for vulnerability detection*, In: Proceedings of the 33rd ACM/IEEE international

conference on automated software engineering. 2018. p. 259-269.

GRIECO, Gustavo, et al. *Echidna: effective, usable, and fast fuzzing for smart contracts*,In: Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis. 2020. p. 557-560.

JIANG, NGUYEN, Tai D., et al. *sfuzz: An efficient adaptive fuzzer for solidity smart contracts*, In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering. 2020. p. 778-788.

MA, Chenyang; SONG, Wei; HUANG, Jeff. *TransRacer: Function Dependence-Guided Transaction Race Detection for Smart Contracts*, In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2023. p. 947-959.

SHOU, Chaofan; TAN, Shangyin; SEN, Koushik. *Ityfuzz: Snapshot-based fuzzer for smart contract*, In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2023. p. 322-333.

Kalra, Sukrit, Goel, Seep, Dhawan, Mohan, Sharma, Subodh.*ZEUS: Analyzing safety of smart contracts*, In: Proceedings 2018 Network and Distributed System Security Symposium. Internet Society, Reston, VA, pp. 1–15.

SO, Sunbeom, et al.*Verismart: A highly precise safety verifier for ethereum smart contracts*, In: 2020 IEEE Symposium on Security and Privacy (SP). IEEE, 2020. p. 1678-1694.

STEPHENS, Jon, et al.*SmartPulse: automated checking of temporal properties in smart contracts*, In: 2021 IEEE Symposium on Security and Privacy (SP). IEEE, 2021. p. 555-571.

WU, Hongjun, et al.*Peculiar: Smart contract vulnerability detection based on crucial data flow graph and pre-training techniques*, In: 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE). IEEE, 2021. p. 378-389.

ZHOU, Qihao, et al.*Vulnerability analysis of smart contract for blockchain-based IoT applications: a machine learning approach*, IEEE Internet of Things Journal, 2022, 9.24: 24695-24707.

YU, Lei, et al.*Pscvfinder: a prompt-tuning based framework for smart contract vulnerability detection*, In: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). IEEE, 2023. p. 556-567.

ZOU, Weiqin, et al. *Smart contract development: Challenges and opportunities*, IEEE transactions on software engineering, 2019, 47.10: 2084-2106.

ZHU, Chenguang, et al. *Identifying solidity smart contract api documentation errors*, Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. 2022.

HAO, Sicheng, et al. *SmartCoCo: Checking Comment-Code Inconsistency in Smart Contracts via Constraint Propagation and Binding*, In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2023. p. 294-306.

TAN, Shin Hwei, et al. , *@tcomment: Testing javadoc comments to detect comment-code inconsistencies*, 2012 IEEE Fifth International Conference on Software Testing,

Verification and Validation. IEEE, 2012. p. 260-269.

PANTHAPLACKEL, Sheena, et al. *Deep just-in-time inconsistency detection between comments and source code*, In: Proceedings of the AAAI Conference on Artificial Intelligence. 2021. p. 427-435.